Product Hub CCN

Documentación técnica para solicitud de permisos de red, ejecución y accesos

---

1. Seguridad y permisos de la aplicación

Qué es

Una aplicación web interna para el equipo de catalogación de CCN. Reemplaza un archivo HTML individual (*product-hub-enhanced.html*) que cada persona abría en su propia máquina y que no permitía guardar histórico ni comparar un reporte contra el anterior.

Para qué sirve

- Dashboard de KPIs de completitud de ficha de producto (nombre, familia, categoría, foto, precio) por sitio/website de venta.
- Detalle de productos filtrado por el KPI que falte (sin foto, sin precio, etc.).
- Comparación entre el reporte de catalogación de hoy y el anterior: qué SKUs son nuevos, cuáles se dieron de baja, y qué campos cambiaron.
- Métricas de completitud por tienda/website.

Quién la usa y cómo se accede

- Uso exclusivo del equipo de catalogación/contenido de CCN.
- Corre en una sola máquina de la empresa; el resto del equipo entra desde su navegador a esa máquina por la red interna (LAN), por IP y puerto (ej. `http://10.x.x.x:8000`).
- No se expone a internet. No requiere ni usa acceso desde fuera de la red de la empresa.
- No tiene login propio (se apoya en que solo es alcanzable dentro de la LAN corporativa).

Qué datos maneja

- El reporte de catálogo que hoy se descarga manualmente de Sales Layer (SKU, nombre, familia, categoría, precio, foto, estado, website, marca). Es información de producto/catálogo, no datos personales ni de clientes.
- Cada carga de un archivo .xlsx queda guardada como snapshot para poder comparar contra el reporte anterior.

Cómo se guardan los datos (sin base de datos)

Por la brecha de seguridad identificada con motores de base de datos, esta versión **no usa ningún motor de base de datos** (ni SQLite, ni Postgres, ni ningún servicio de este tipo). Cada carga de reporte

se guarda como un archivo .json plano en el disco local de la máquina donde corre el servidor, dentro de la carpeta de usuario (`~/.local/product_hub_data/`). No hay puerto de base de datos abierto, no hay motor corriendo, no hay conexión de red asociada a almacenamiento: es lectura/escritura de archivos igual que cualquier script que guarda un .xlsx o .csv en disco.

Arquitectura técnica (resumen)

Componente	Detalle
Backend	Python (FastAPI), corre localmente en la máquina designada.
Frontend	React, servido por el mismo proceso backend (un solo ejecutable/proceso, un solo puerto).
Dependencias externas	Ninguna en tiempo de ejecución. No llama a servicios de internet, no envía datos a terceros.
Persistencia	Archivos JSON en disco local (ver sección anterior).

Plan futuro (fuera del alcance de este permiso)

Más adelante se evalúa conectar la API de Sales Layer para traer la data automáticamente en vez de subir el .xlsx a mano. Eso implicaría una llamada saliente a un servicio externo (Sales Layer) con su propia credencial/API key, y se documentará y solicitará como un permiso aparte cuando se vaya a implementar.

2. Justificación de herramientas y accesos solicitados

Detalle de para qué se necesita cada herramienta marcada en el formulario de acceso (Terminal PowerShell, Python, Node.js), en el marco del proyecto descrito en la sección 1.

Terminal PowerShell

Para qué se usa: Ejecutar los comandos que levantan y detienen la aplicación en la máquina donde va a correr: arrancar el servidor backend (`uvicorn`), instalar dependencias de Python y de Node, y generar el build del frontend cuando haya cambios.

Alcance: Solo comandos del propio proyecto (instalar paquetes, correr el servidor, compilar el frontend). No se usa para administración del sistema, no requiere privilegios de administrador para el día a día, y no se ejecuta nada de forma remota.

Python

Para qué se usa: Es el motor que corre el backend de la aplicación (framework FastAPI). Es lo que procesa el reporte .xlsx que se sube, calcula los KPIs y responde a las peticiones del dashboard.

Alcance: Se instala en un entorno virtual dedicado al proyecto (aislado del resto del sistema), fuera de la carpeta de OneDrive. No se conecta a ningún servicio externo; todas sus dependencias (FastAPI, pandas, openpyxl) son librerías que corren localmente.

Node.js

Para qué se usa: Es el motor necesario para instalar (`npm install`) y compilar (`npm run build`) el frontend, escrito en React. Sin Node.js no se puede generar el sitio web que ve el usuario final; una vez compilado, el propio backend en Python lo sirve, así que Node.js solo hace falta durante el desarrollo y cuando se actualiza el frontend, no en el uso diario de la aplicación por el equipo.

Alcance: Se usa únicamente para descargar paquetes públicos de React/Vite (los mismos que usa cualquier proyecto web moderno) y compilar el código local del proyecto. No corre como servicio expuesto a la red.

Alcance general de la solicitud

Todo el proyecto se ejecuta de manera local, en una sola máquina de la empresa, sin exposición a internet y sin conexión a servicios externos. Las tres herramientas anteriores son necesarias únicamente para poder instalar, compilar y arrancar la aplicación; ninguna se usa para tareas fuera del alcance de este proyecto.